

Face Recognition Using Python and OpenCV

Guided by: Mr. Partha Pratim Paul
Dept of Information Technology
Asansol Engineering College
West Bengal, India

Amlan Dutta
Dept of Information Technology
Asansol Engineering College, India
Email: amlan.it.aec432@gmail.com

Hena Kouser
Dept of Information Technology
Asansol Engineering College, India
Email: hena.it.aec@gmail.com

Nandita Kumari
Dept of Information of Technology
Asansol Engineering College India
Email:- nandita.it.aec@email.com

Anusua Singha
Dept of Information of Technology
Asansol Engineering College India
Email:- anusua.it.aec@email.com

Abstract: Highly precise and effective face recognition systems have been made possible by the quick development of computer vision and machine learning techniques. With an emphasis on real-time facial detection and recognition, this paper shows a face recognition system constructed with Python and the OpenCV package. Using pre-trained models for reliable face identification, the system uses the Haar Cascade Classifier for face detection. Utilising the Local Binary Patterns Histograms (LBPH) algorithm for feature extraction and classification is a crucial part of the system, as it improves facial recognition performance and accuracy in a variety of lighting scenarios and viewpoints.

1.Introduction: Because of its many uses, ease of use, and non-intrusiveness, face recognition has become one of the most popular biometric identification methods. Face recognition is essential for identity verification, improving security, and enabling smooth machine interaction in anything from security systems to customised user experiences. The accuracy and resilience of face recognition systems have greatly increased with developments in computer vision, artificial intelligence (AI), and machine learning. The advent of robust libraries like OpenCV, which streamline challenging image processing and machine learning tasks, has made it easier to design such systems. The creation of a face recognition system with Python and the OpenCV package is the main goal of this project. Python has emerged as a leading tool in the fields of computer vision and machine learning thanks to its robust library ecosystem and straightforward syntax. In contrast, OpenCV is a comprehensive open-source computer vision library that offers a wide range of tools and algorithms for computer vision tasks, such as face identification and recognition, pattern recognition, and real-time picture processing.

2.Motivation: The growing need for better security, convenience, and automation in today's digital environment is what motivated the development of a face recognition system utilising Python and OpenCV. Face recognition is a biometric technique that has gained popularity because of its high accuracy, convenience of use, and non-intrusive nature. It makes it possible to automatically identify and validate people using their distinct face features, which makes it an effective tool for a variety of uses.

3.Scope: The scope of the Face Recognition System using Python and OpenCV extends across a broad range of industries and applications. Surveillance systems make heavy use of face recognition technology to instantly identify or authenticate people. Law enforcement organisations, for instance, utilise facial recognition technology in public safety to keep an eye on crowds, find wanted offenders, or stop illegal access to sensitive locations. Secure access control systems are using face recognition more and more. Face recognition technology can be used in place of conventional ID cards, PINs, or passwords in banks, hospitals, and corporate offices to provide access to restricted areas. Similar to this, face recognition is used in

computers and smartphones to unlock them and verify users for a number of services, including secure apps and payment processing. It makes the user experience more comfortable and secure by doing away with the requirement for manual input or physical tokens.

4.Methodologies: The system uses images stored in a specified folder to recognize faces captured by a webcam and mark attendance in a CSV file. Here is a step-by-step breakdown of the methodology:

4.1. Data Preparation: Load Training Images

The code starts by loading images from a directory (Person_Images), which contains images of different individuals. It initializes two lists: images (to store the loaded images) and class Names (to store the corresponding names of the individuals extracted from the image file names).

4.2. Encoding Training Images A function findEncodings(images) is defined to encode all the training images. Face Encoding: For each image, it converts the image from BGR (used by OpenCV) to RGB (used by the face recognition library) and then calculates the face encodings using face_recognition.face_encodings(img).The function checks if the face encoding exists (to handle cases where no face is detected) and appends the first encoding to encodeList, which is later returned as encoded_face_train.

4.3. Attendance Marking Function A function markAttendance(name) is created to log attendance details. It opens the Attendance.csv file in read-plus mode ('r+'). The function reads existing entries into a list (myDataList) and checks if the given name is already present. If not, it records the current time and date using Python's datetime module and writes them into the CSV file.

4.4. Capture Video Stream from Webcam The code initializes the webcam feed using cv2.VideoCapture(0).It enters a loop where it continuously captures frames from the webcam.

4.5. Real-Time Face Recognition Preprocessing: Each frame is resized to 1/4th of its original size to speed up processing, and converted from BGR to RGB.Face Detection and Encoding: It uses face_recognition. face_locations(imgS) to detect faces in the current frame and then encodes them using face_recognition.face_encodings(imgS, faces_in_frame).

Face Comparison:For each detected face, it compares the encoded face with the pre-computed encodings from the training images using

face_recognition.compare_faces(encoded_face_train, encode_face).

It calculates the face distances using face_recognition.

face_distance(encoded_face_train, encode_face) and identifies the closest match (smallest distance).

4.6. Draw Rectangle and Display Name If a match is found, it retrieves the name corresponding to the matched face and draws a rectangle around the detected face in the frame. It also displays the person's name using cv2.putText().

4.7. Mark Attendance If the face is recognized, the markAttendance(name) function is called to log the attendance.

4.8. Display the Webcam Feed The processed frames with rectangles and names are displayed in a window titled 'webcam'. The loop continues until the 'q' key is pressed to exit.

4.9. Exit and CleanupThe webcam feed is released and all OpenCV windows are closed when the loop breaks.

5.Tools and Technologies

Python: Face recognition in Python involves using libraries like face_recognition, which simplify the process of detecting and identifying faces in images. You typically start by loading images and then applying face recognition models to locate and recognize faces. The results can be processed to determine which faces match known identities

CSV: To manage and store face recognition data, you might use CSV files. In this context, a CSV file would typically contain records with identifiers or names alongside corresponding face encoding data, which represents the unique features of each face. This makes it easy to track and match faces across different images or sessions.

6.Process and Explanation The system is designed to recognize faces in real-time using a webcam and mark attendance based on matching faces from a pre-trained dataset. Here's how the process unfolds step by step:

6.1.Loading and Training the system

The first step in the process involves preparing the system with known faces. This is a one-time setup:

- **Image Collection:** A group of pictures of well-known people, including staff members or pupils, are collected in a particular folder. Every picture

depicts a human whose face the system ought to be able to identify.

- **Encoding Faces:** After processing these photos, the algorithm extracts each person's distinct facial traits. This is accomplished through the use of a method known as face encoding, which transforms the facial traits into a vector—a mathematical representation—that can subsequently be compared. For later usage, these encodings are saved.

- **Keeping Names:** In addition to the encodings, the names of the people (which were taken from the image filenames) are also kept.

6.2. Real-time Face Detection and Recognition

Once the system is set up, it moves into the real-time face recognition phase, where it continuously processes webcam frames to detect and recognize faces:

- **Webcam Feed Capture:** The system initiates a webcam video stream and records individual frames.

- **Face Detection:** The system uses face detection algorithms to identify the faces in each frame. This makes it easier to spot the areas of the picture that include faces.

- **Face Encoding:** After faces are identified, the system creates encodings for each one, just like it did during training.

6.3. Face Comparison and Matching

Now that the system has captured the faces from the webcam, it proceeds to compare them with the stored encodings from the training images:

- **Face Comparison:** The system compares the encodings of the faces it has identified with the encodings it has already saved (from photos of persons it knows). The system uses a technique known as face comparison to accomplish this, determining whether the identified face resembles any of the faces that have already been trained.

- **Face Matching:** The system determines the distance between the encodings in order to get the closest match. The faces are increasingly alike the closer they are to one another. The technology can identify the individual in the picture if it detects a match.

6.4. Marking Attendance

If a match is found, the system marks the attendance of the recognized individual:

- **Drawing a Box Around the Face:** The method surrounds the face in the frame with a rectangular box to visually verify the recognition.

- **Name Display:** In addition, the system shows the identified person's name on the frame, typically beneath their face.

- **Attendance Recording:** If the individual hasn't been recorded as present already, the system then logs their name, the date, and the current time in an attendance log (CSV file).

6.5. Displaying the Results

The video feed with the identified face and name is shown on the screen in real time while the system is operating. This enables the operator or user to view which people are being acknowledged along with their attendance status.

6.6. Exiting the System

The system operates constantly for a predetermined amount of time (for example, 20 seconds), after which it either automatically ceases or can be manually stopped. The system stops taking frames by:

- Releasing the webcam.
- Closes every window on the screen to free up resources.

Visual Flow of the Process:

1. Setup Phase (Once):

- Collect images of known individuals and store them.
- Generate encodings of these images to be used later for comparison.

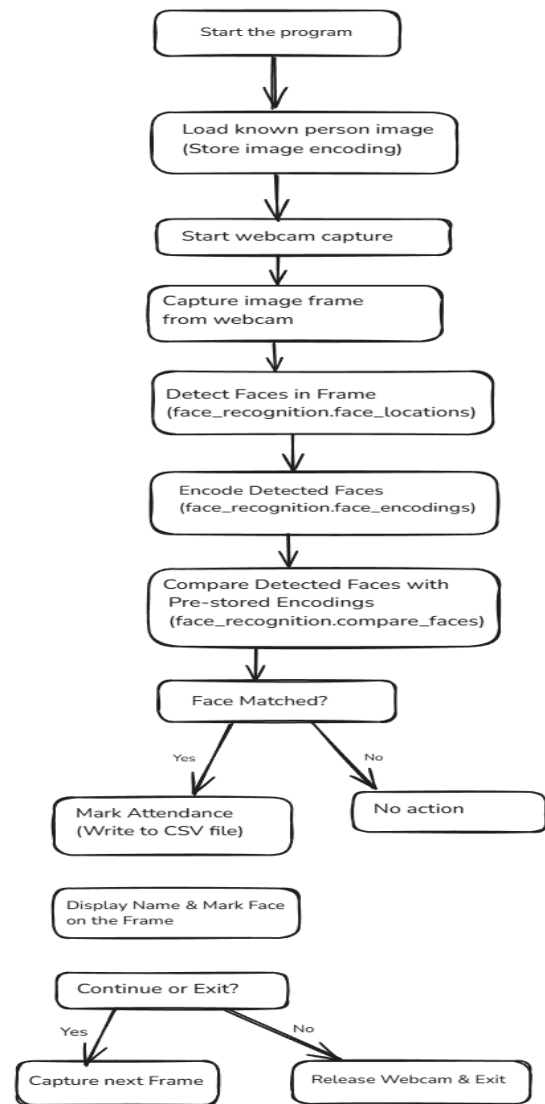
2. Recognition Phase (Real-time):

- Capture video frames from the webcam.
- Detect faces in the frames.
- Generate encodings for each detected face.
- Compare the detected faces with pre-trained encodings.

- Match found?
 - If yes:
 - Display a rectangle around the recognized face.
 - Show the name of the recognized person on the screen.
 - Record the attendance (name, time, date) in a file.
 - If no:
 - No action is taken, and the process continues to the next frame.

3. Exit:

- Stop the video capture after a set duration or when the user chooses to quit.
- Release resources (close webcam and windows).



Key Components of the System:

- Face Detection: Identifying faces in the webcam feed.
- Face Encoding: Converting detected faces into mathematical representations.
- Face Comparison: Comparing the captured face to stored encodings to find a match.
- Attendance Logging: Storing attendance data (name, time, and date) when a match is found.
- Real-time Feedback: Displaying a live video feed with overlaid information (face bounding box and name).

7. Expected Output

7.1. Face Identification in Real-Time

- Live Video Feed: The primary output is a live video feed from a webcam (or other video source) where detected faces are highlighted.
- Face Boundaries: Faces detected in the video feed are outlined with rectangles. These rectangles are drawn around the face in the video frame.
- Names Displayed: Above or below each detected face, the system displays a label with the recognized person's name. If the face isn't recognized, it typically displays "Unknown."

7.2. Console Outputs and Logs

- Recognition Results: The console or

terminal may display logs or print statements showing the recognition results, including the names of detected individuals or "Unknown" if no match is found.

- Error Messages: Any issues encountered, such as missing files or processing errors, might be logged or displayed as error messages.

7.3. Additional Outputs

- CSV Data Integration: The system reads from the CSV file to match faces, so you should ensure the CSV file is correctly formatted and includes the expected data (image filenames and names).
- Updated CSV: In more advanced implementations, you might write additional functionality to update the CSV with new data (e.g., adding new faces or updating names).

8. Conclusion In conclusion, a simple face recognition system using Python and CSV effectively demonstrates the fundamentals of facial recognition technology. By leveraging Python libraries such as `face\_recognition` and `opencv-python`, and storing data in a CSV file, the system can identify and verify individuals through real-time video feeds. While it provides a practical introduction and serves various applications like user authentication and photo management, its limitations in scalability and accuracy highlight the need for more advanced techniques in complex scenarios. Overall, this project offers a solid foundation for further exploration into more sophisticated face recognition systems.

9. Acknowledgement We would like to sincerely thank our professor Mr Partha Pratim Paul sir for guiding us throughout this project work also would like to thank our other faculty members from the computer engineering department at Asansol Engineering College for allowing us to perform our project work.

10. References

[1] Face Detection in Real Time Based on HOG. N. J. Wang, S. C. Chang and P. J. Chou. Taipei, Taiwan: IEEE, DOI:10.1109/ISPACS.2012.6473506, 2012. International Symposium on Intelligent Signal Processing and Communications Systems. pp. 333-337. ISBN: 978-1-4673 5081-5.

[2] Face Recognition Homepage, <http://www.face-rec.org/algorithms/>

[3] Seeing with OpenCV, Article, http://www.cognotics.com/opencv/servo_2007_series/part_1/index.html

[4] Python with deep learning, <https://www.tutorialspoint.c>

[5] Face Detection and Tracking using OpenCV. S.V.Viraktamath, Mukund Katti, Aditya Khatawkar, Pavan Kulkarni. 3, s.l.: SIJ, July-August 2013, The Standard International Journals (The SIJ) , Vol. 1, pp. 45-50. ISSN: 2321 – 2403